

Connected to Python

```
In [ ]: import numpy as np
import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import warnings
import pmdarima as pm # para halar el mejor modelo de ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox # para la prueba de auto
warnings.filterwarnings('ignore')
from statsmodels.tsa.stattools import adfuller # para el test de raiz unitaria
from statsmodels.stats.diagnostic import het_arch
from statsmodels.stats.stattools import jarque_bera
from scipy.stats import norm
import scipy.stats as stats
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
In [ ]: CARPETA_ACTUAL = Path('.').resolve()

# 2. .parent sube un nivel (sale de 'codigo') y luego entra a 'bases_de_datos'
ruta_hipotecas = CARPETA_ACTUAL.parent / "bases_de_datos" / "Hipotecas_MORTGAGE3

# 3. Carga la base de datos
df_hipotecas = pd.read_csv(ruta_hipotecas, parse_dates=['observation_date'], ind
```

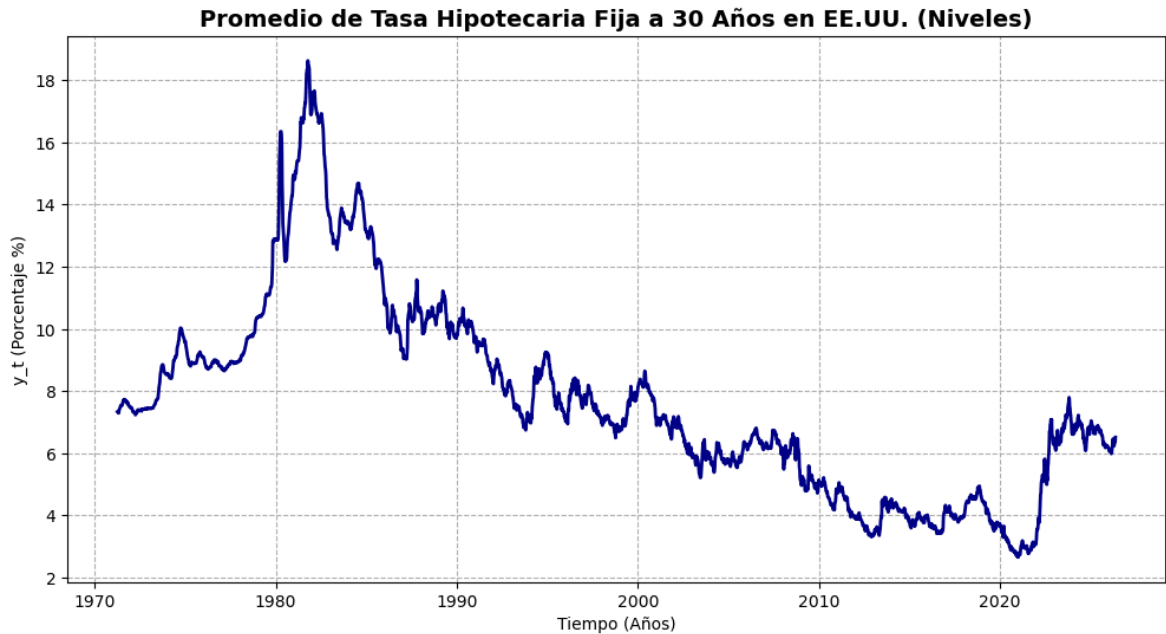
```
In [ ]: # Renombro las columnas por que tiene un nombre muy feo y tedioso
df_hipotecas = df_hipotecas.rename(columns={'MORTGAGE30US': 'Tasa_hipotecaria'})
print(df_hipotecas.info())
print(df_hipotecas.describe())
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 2878 entries, 1971-04-02 to 2026-05-21
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Tasa_hipotecaria  2878 non-null   float64
dtypes: float64(1)
memory usage: 45.0 KB
None
```

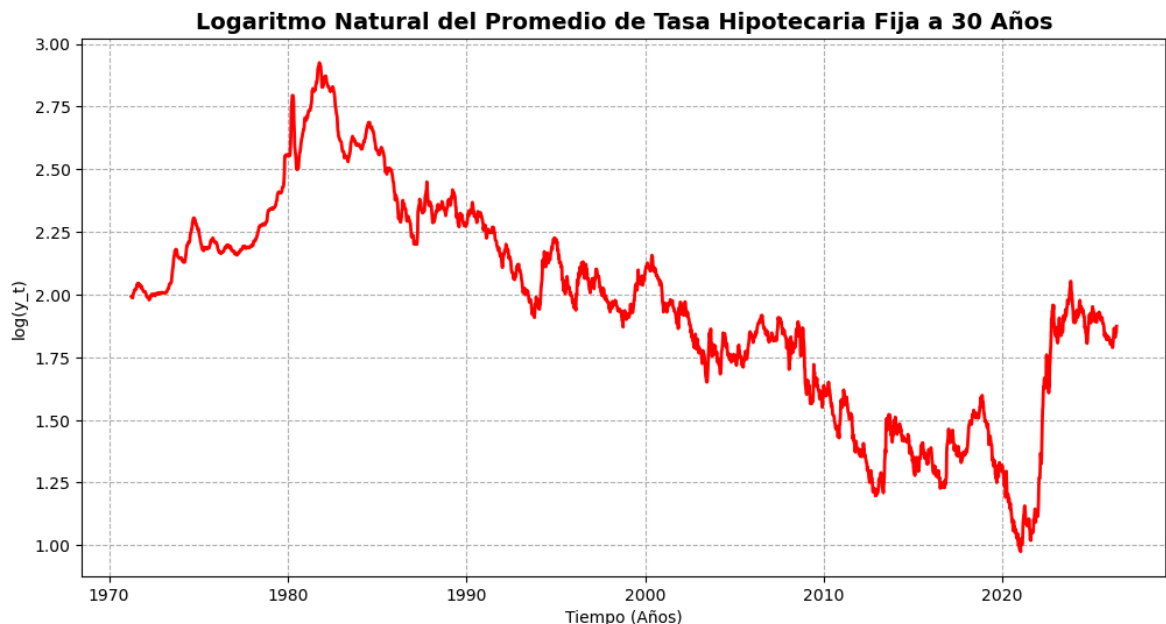
	Tasa_hipotecaria
count	2878.000000
mean	7.687224
std	3.187367
min	2.650000
25%	5.472500
50%	7.230000
75%	9.237500
max	18.630000

```
In [ ]: plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['Tasa_hipotecaria'], color='darkblue',
plt.title('Promedio de Tasa Hipotecaria Fija a 30 Años en EE.UU. (Niveles)', fon
plt.xlabel('Tiempo (Años)')
plt.ylabel('y_t (Porcentaje %)')
plt.grid(True, linestyle='--')
plt.show()
```

Nota : Se puede apreciar que no tiene ni media ni varianza constante, por lo que
 # Por ello voy a aplicar primer logaritmo y luego diferencia para intentar hacer



```
In [ ]: df_hipotecas['log_hipotecas'] = np.log(df_hipotecas['Tasa_hipotecaria'])
# Se añade una columna de log de los datos en la base de datos
plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['log_hipotecas'], color='red', linewidth=2)
plt.title('Logaritmo Natural del Promedio de Tasa Hipotecaria Fija a 30 Años', fontweight='bold')
plt.xlabel('Tiempo (Años)')
plt.ylabel('log(y_t)')
plt.grid(True, linestyle='--')
plt.show()
```



```
In [ ]: plt.figure(figsize=(12, 6))

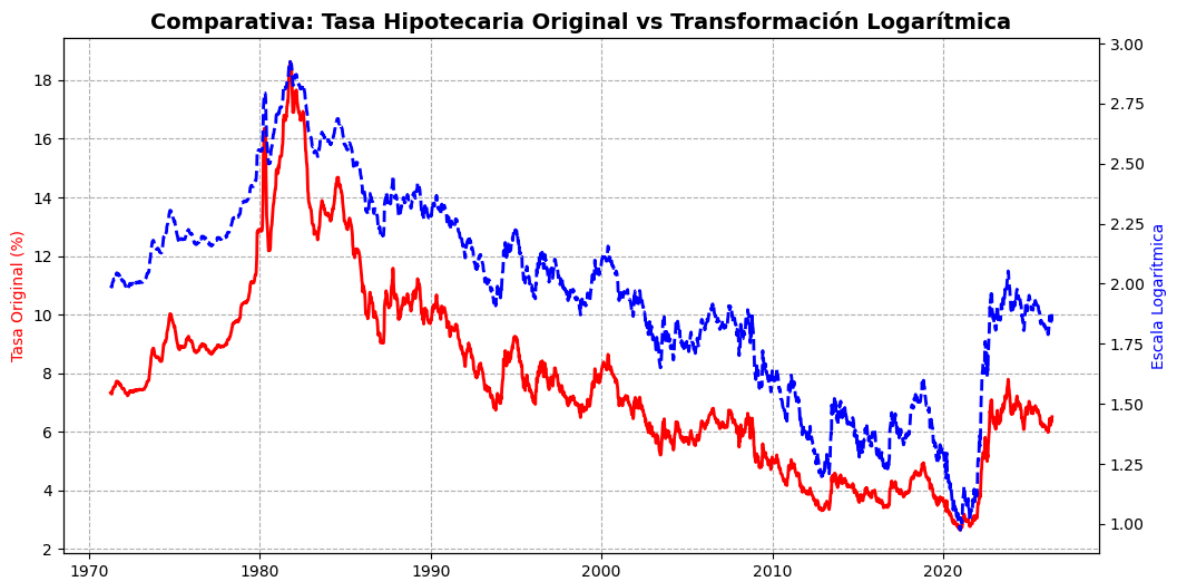
# 1. Grafica serie original (Rojo)
plt.plot(df_hipotecas.index, df_hipotecas['Tasa_hipotecaria'], color='red', linewidth=2)
plt.ylabel('Tasa Original (%)', color='red')
plt.grid(True, linestyle='--')
```

```
# 2. Activar el segundo eje Y compartiendo el mismo eje X
plt.twinx()

# 3. Grafica serie en Logaritmo (Azul)
plt.plot(df_hipotecas.index, df_hipotecas['log_hipotecas'], color='blue', linewidth=2)
plt.ylabel('Escala Logarítmica', color='blue')

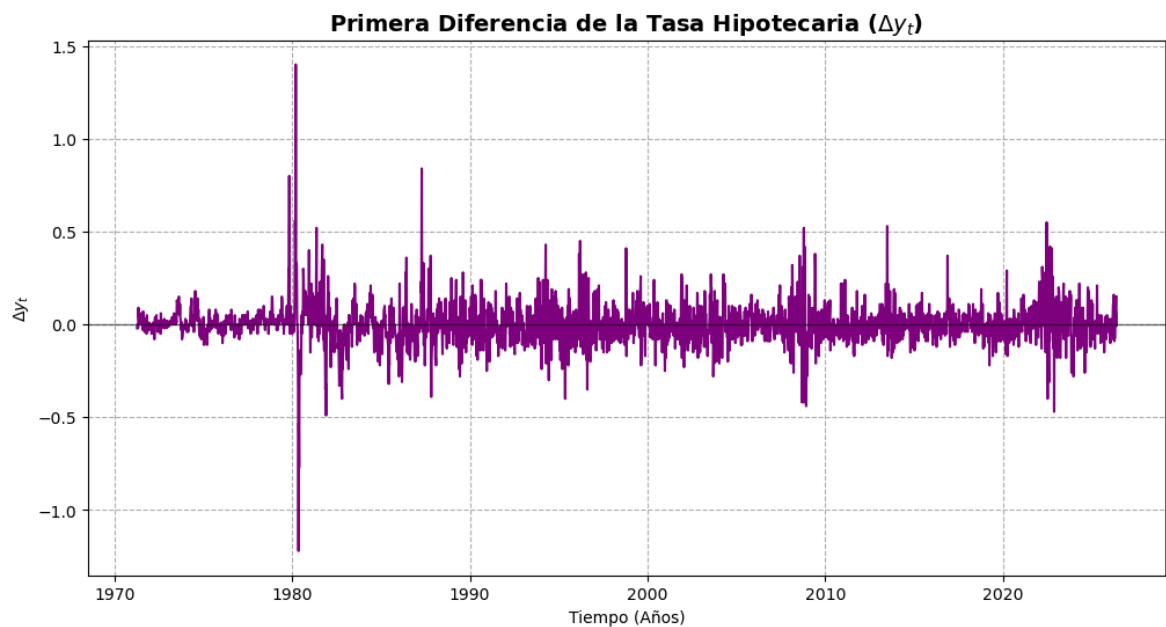
# Titulos
plt.title('Comparativa: Tasa Hipotecaria Original vs Transformación Logarítmica')
plt.xlabel('Tiempo (Años)')

plt.show()
#se comportan bastante igual por ello no creo que sea necesario aplicar Logaritmo
```



```
In [ ]: # Se añade la columna de diferencia a la base de datos
df_hipotecas['Dif_hipotecas'] = df_hipotecas['Tasa_hipotecaria'].diff()

plt.figure(figsize=(12, 6))
plt.plot(df_hipotecas.index, df_hipotecas['Dif_hipotecas'], color='purple', line
plt.title('Primera Diferencia de la Tasa Hipotecaria ($\Delta y_t$)', fontsize=12)
plt.xlabel('Tiempo (Años)')
plt.ylabel('$\Delta y_t$')
plt.axhline(0, color='black', linestyle='--', alpha=0.5, linewidth=1) # Línea de
plt.grid(True, linestyle='--')
plt.show()
# mejora :D entonces creo que por el momento es un ARIMA(?,1,?)
#recapitulando no es necesario aplicar Logaritmo pero si aplicar diferencia
```



```
In [ ]: # Toca quitar el primer dato por la diferencia
serie_diferenciada = df_hipotecas['Dif_hipotecas'].dropna()

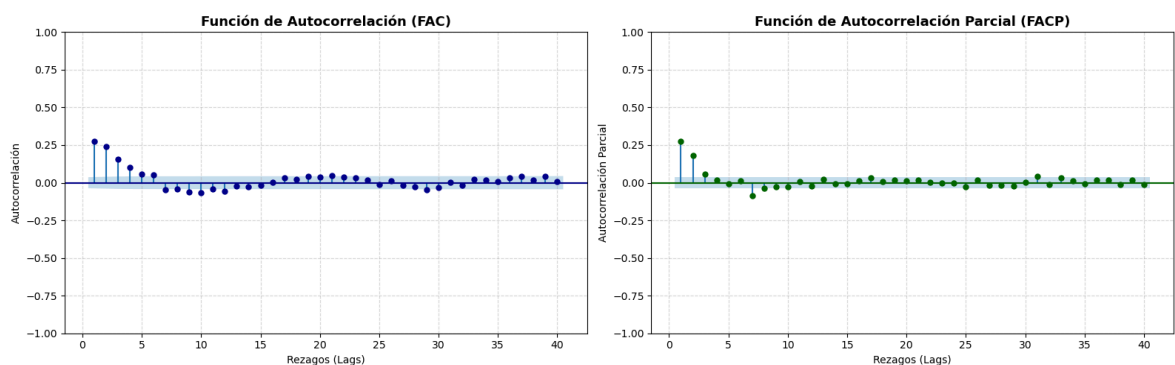
plt.figure(figsize=(16, 5))

#Grafico FAC
plt.subplot(1, 2, 1)
plot_acf(serie_diferenciada, lags=40, zero=False, ax=plt.gca(), color='darkblue')
plt.title('Función de Autocorrelación (FAC)', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)

#Grafico FACP
plt.subplot(1, 2, 2)
plot_pacf(serie_diferenciada, lags=40, zero=False, ax=plt.gca(), color='darkgreen')
plt.title('Función de Autocorrelación Parcial (FACP)', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación Parcial')
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

# yo diria que puede ser un ARIMA(2,1,2) por parcimonia ya que tanto
# la parte autoregresiva como la de media movil parecieran que tiene solucion im
```



```
In [ ]: modelo_arima_212 = ARIMA(df_hipotecas['Tasa_hipotecaria'], order=(2, 1, 2))
resultados_212 = modelo_arima_212.fit()
```

```
print(resultados_212.summary())
```

```
SARIMAX Results
=====
Dep. Variable:      Tasa_hipotecaria      No. Observations:      2878
Model:              ARIMA(2, 1, 2)         Log Likelihood          2463.444
Date:               Sun, 24 May 2026       AIC                     -4916.889
Time:               15:22:44               BIC                     -4887.066
Sample:             0                      HQIC                   -4906.139
                    - 2878
Covariance Type:    opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          1.6520       0.039      42.484      0.000       1.576       1.728
ar.L2         -0.7597       0.027     -28.552      0.000      -0.812      -0.708
ma.L1         -1.4533       0.041     -35.862      0.000      -1.533      -1.374
ma.L2          0.6145       0.024      25.546      0.000       0.567       0.662
sigma2          0.0106       0.000      86.723      0.000       0.010       0.011
=====
==
Ljung-Box (L1) (Q):              0.17      Jarque-Bera (JB):              26970.
30
Prob(Q):              0.68      Prob(JB):              0.
00
Heteroskedasticity (H):          0.86      Skew:              0.
75
Prob(H) (two-sided):          0.02      Kurtosis:              17.
92
=====
==

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
In [ ]: # Aplicamos el test a la serie diferenciada (eliminando los valores nulos)
serie_test = df_hipotecas['Dif_hipotecas'].dropna()
resultado_adf = adfuller(serie_test)

# resultados del test
print(f'Estadístico ADF: {resultado_adf[0]:.4f}')
print(f'p-valor: {resultado_adf[1]:.4f}')
print('Valores Críticos:')
for clave, valor in resultado_adf[4].items():
    print(f'    {clave}: {valor:.4f}')

# conclusion automatica
if resultado_adf[1] < 0.05:
    print("Se rechaza H0; la serie es estacionaria.")
else:
    print("No se rechaza H0; la serie no es estacionaria.")
```

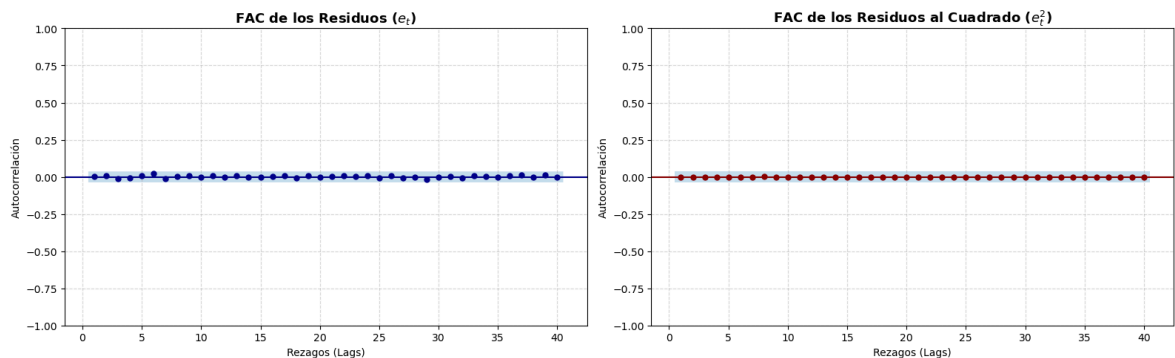
```
Estadístico ADF: -17.2567
p-valor: 0.0000
Valores Críticos:
    1%: -3.4326
    5%: -2.8625
    10%: -2.5673
Se rechaza H0; la serie es estacionaria.
```

```
In [ ]: # se sacan los residuos del modelo :D
residuos = resultados_212.resid
residuos_cuadrado = residuos ** 2

# para que entren los dos graficos en la misma figura
plt.figure(figsize=(16, 5))

# FAC de los residuos
plt.subplot(1, 2, 1)
plot_acf(residuos, lags=40, zero=False, ax=plt.gca(), color='darkblue')
plt.title('FAC de los Residuos ( $e_t$ )', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)

# fac de los residuos al cuadrado
plt.subplot(1, 2, 2)
plot_acf(residuos_cuadrado, lags=40, zero=False, ax=plt.gca(), color='darkred')
plt.title('FAC de los Residuos al Cuadrado ( $e_t^2$ )', fontsize=13, fontweight='bold')
plt.xlabel('Rezagos (Lags)')
plt.ylabel('Autocorrelación')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



```
In [ ]: ljung_box = acorr_ljungbox(
    residuos,
    lags=[6, 12, 18, 24],
    return_df=True
)

print("Prueba Ljung-Box")
print(ljung_box)

if (ljung_box['lb_pvalue'] > 0.05).all():
    print("No se rechaza H0; no hay autocorrelación en los residuos.")
else:
    print("Se rechaza H0; hay autocorrelación en los residuos.")
# H0: no hay autocorrelación en los residuos.
# Si p-value > 0.05, no se rechaza H0.
```

Prueba Ljung-Box

	lb_stat	lb_pvalue
6	2.287843	0.891415
12	3.101020	0.994788
18	3.683675	0.999870
24	4.326997	0.999997

No se rechaza H0; no hay autocorrelación en los residuos.

```
In [ ]: arch_test = het_arch(residuos, nlags=12)

print("Prueba ARCH")
print("LM statistic:", arch_test[0])
print("LM p-value:", arch_test[1])

# H0: no hay efectos ARCH.
# Si p-value > 0.05, no se rechaza H0.
```

Prueba ARCH
LM statistic: 390.7001813250866
LM p-value: 3.5206403649889906e-76

```
In [ ]: arch_test = het_arch(residuos, nlags=12)

print("Prueba ARCH")
print("LM statistic:", arch_test[0])
print("LM p-value:", arch_test[1])

if arch_test[1] > 0.05:
    print("No se rechaza H0; no hay efectos ARCH.")
else:
    print("Se rechaza H0; hay efectos ARCH.")

# H0: no hay efectos ARCH.
# Si p-value > 0.05, no se rechaza H0.
```

Prueba ARCH
LM statistic: 390.7001813250866
LM p-value: 3.5206403649889906e-76
Se rechaza H0; hay efectos ARCH.

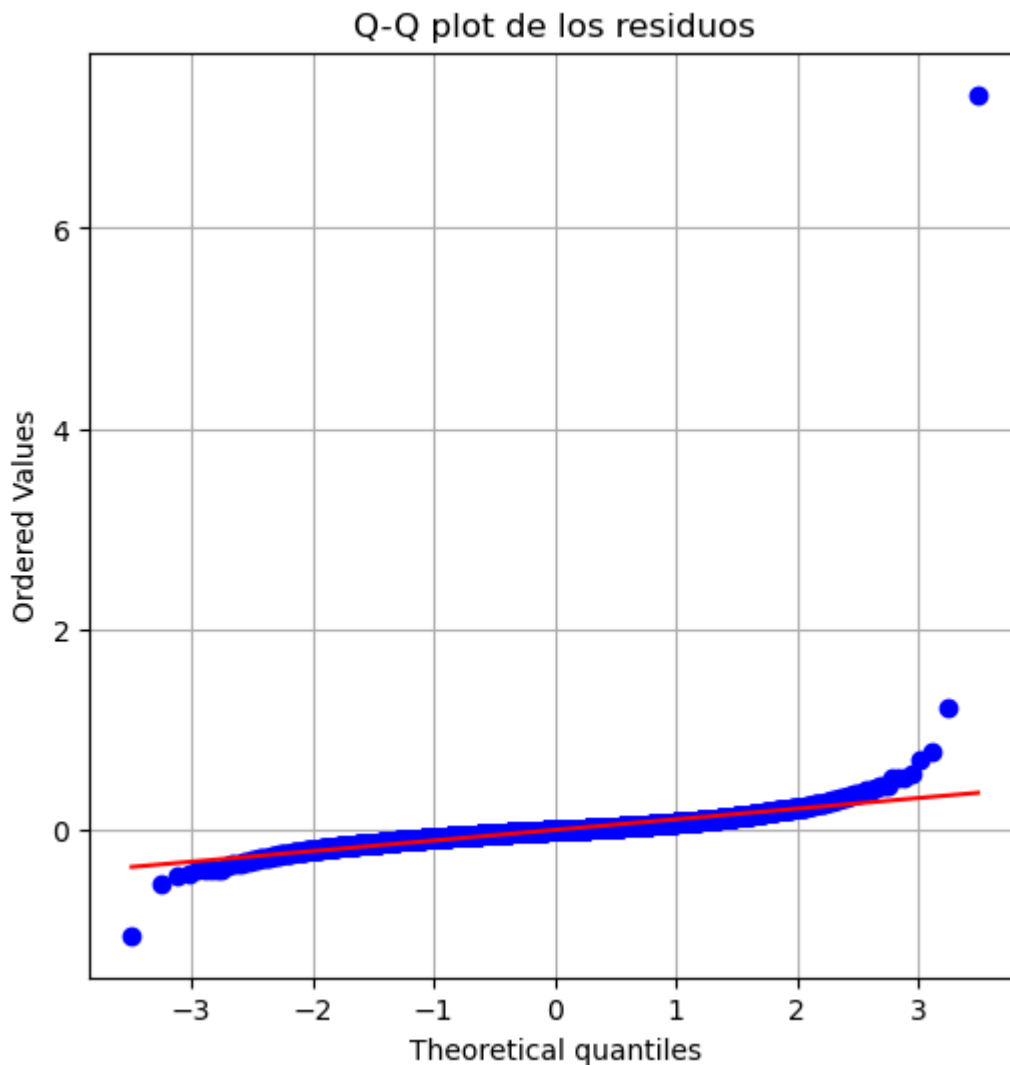
```
In [ ]: jb_stat, jb_pvalue, asimetria, curtosis = jarque_bera(residuos)

print("Prueba Jarque-Bera")
print("Jarque-Bera statistic:", jb_stat)
print("Jarque-Bera p-value:", jb_pvalue)

if jb_pvalue > 0.05:
    print("No se rechaza H0; los residuos siguen una distribución normal.")
else:
    print("Se rechaza H0; los residuos no siguen una distribución normal.")
# H0: Los residuos siguen una distribución normal.
# Si p-value > 0.05, no se rechaza H0.
```

Prueba Jarque-Bera
Jarque-Bera statistic: 165209371.2097154
Jarque-Bera p-value: 0.0
Se rechaza H0; los residuos no siguen una distribución normal.

```
In [ ]: plt.figure(figsize=(6, 6))
stats.probplot(residuos, dist="norm", plot=plt)
plt.title("Q-Q plot de los residuos")
plt.grid(True)
plt.show()
```



```
In [ ]: # 1. Extraemos Los residuos del modelo estimado
residuos = resultados_212.resid

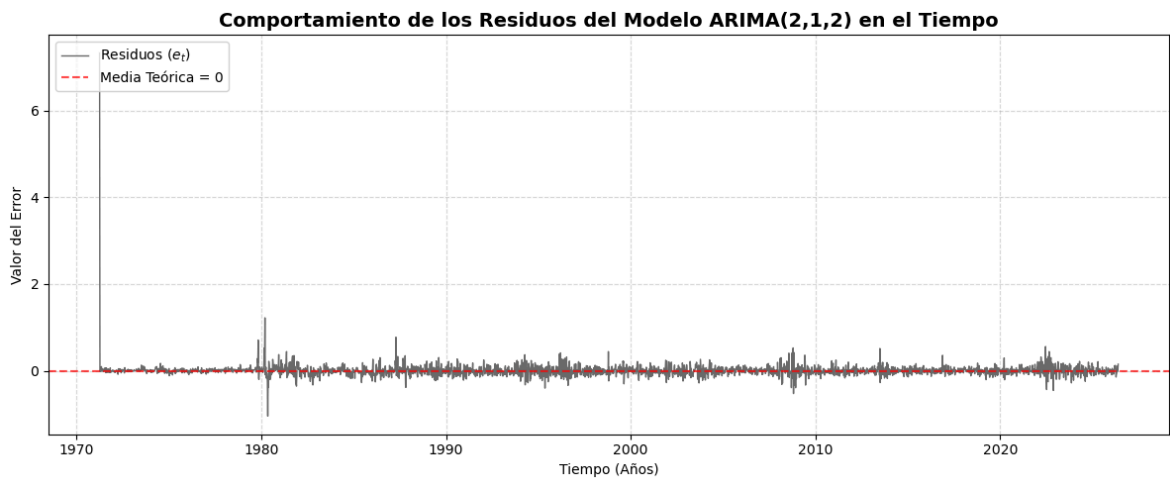
plt.figure(figsize=(12, 5))

# 2. Graficamos Los residuos usando el índice de fechas de tu DataFrame
plt.plot(df_hipotecas.index, residuos, color='darkwhite' if False else 'dimgray')

# 3. Añadimos una línea horizontal en cero como referencia obligatoria
plt.axhline(0, color='red', linestyle='--', alpha=0.7, linewidth=1.5, label='Med

# Configuración estética para el póster
plt.title('Comportamiento de los Residuos del Modelo ARIMA(2,1,2) en el Tiempo',
plt.xlabel('Tiempo (Años)')
plt.ylabel('Valor del Error')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='upper left')

plt.tight_layout()
plt.show()
```



```
In [ ]: df_hipotecas = df_hipotecas.asfreq(df_hipotecas.index.inferred_freq)
```

```
# 2. Configurar y entrenar el modelo final directamente en NIVELES
modelo_final = SARIMAX(
    df_hipotecas['Tasa_hipotecaria'],
    order=(2, 1, 2),
    trend='c',
    enforce_stationarity=False,
    enforce_invertibility=False
)
resultado_final = modelo_final.fit(dispatch=False)

# 3. Proyectar 10 periodos hacia el futuro con fechas correctas
pasos_futuros = 10
pronostico_objeto = resultado_final.get_forecast(steps=pasos_futuros)

# 4. Extraer la media y los intervalos
pronostico_nivel = pronostico_objeto.predicted_mean
intervalos_nivel = pronostico_objeto.conf_int()

# 5. Imprimir tabla de control en consola
tabla_pronostico = pd.DataFrame({
    "Pronóstico (%)": pronostico_nivel,
    "Límite Inferior": intervalos_nivel.iloc[:, 0],
    "Límite Superior": intervalos_nivel.iloc[:, 1]
})
print("\n--- TABLA DE PRONÓSTICOS GENERADA CON ÉXITO ---")
print(tabla_pronostico.round(4))
```

```
--- TABLA DE PRONÓSTICOS GENERADA CON ÉXITO ---
```

	Pronóstico (%)	Límite Inferior	Límite Superior
2026-05-22	6.5225	6.4819	6.5631
2026-05-23	6.5315	6.4597	6.6032
2026-05-24	6.5364	6.4312	6.6417
2026-05-25	6.5399	6.4056	6.6741
2026-05-26	6.5418	6.3807	6.7029
2026-05-27	6.5431	6.3578	6.7284
2026-05-28	6.5438	6.3363	6.7514
2026-05-29	6.5443	6.3162	6.7725
2026-05-30	6.5446	6.2973	6.7919
2026-05-31	6.5447	6.2796	6.8099